

Tugas Akhir Semester 3

Pemrograman Berbasis Objek



Sistem Manajemen Rental Mobil

25040750004 Alifya Putri Aisyka

24040700107 Mohammad Daffa Shafy Loggery

Jurnal: Sistem Manajemen Rental Mobil

Judul Proyek: Sistem Manajemen Rental Mobil Menggunakan Object-Oriented Programming (C++)

Proyek ini menyajikan Sistem Manajemen Rental Mobil yang dirancang menggunakan prinsip Object-Oriented Programming (OOP) dalam bahasa C++. Sistem ini mengatasi kebutuhan akan manajemen rental kendaraan yang efisien dalam bisnis rental mobil skala kecil hingga menengah. Dengan mengimplementasikan konsep inti OOP seperti classes, objects, encapsulation, dan composition, sistem ini menyediakan pendekatan terstruktur untuk mengelola inventaris mobil, informasi pelanggan, dan transaksi rental. Solusi ini menawarkan fitur fundamental termasuk pendaftaran mobil, pelacakan ketersediaan, manajemen pelanggan, dan pemrosesan transaksi dengan pembuatan struk.

Isu Utama dan Pernyataan Masalah

Salah satu tantangan utama dalam manajemen bisnis rental mobil adalah mempertahankan sistem terorganisir untuk melacak ketersediaan kendaraan, informasi pelanggan, dan transaksi rental, terutama selama periode permintaan tinggi. Pendekatan tradisional—seperti menggunakan catatan kertas atau spreadsheet dasar—tidak efisien, rentan terhadap kesalahan manusia, dan sulit untuk diskala.

Masalah Inti yang Diatasi:

Ketiadaan sistem yang sederhana, andal, dan otomatis untuk mengelola operasi rental mobil di bisnis skala kecil hingga menengah. Banyak perusahaan rental tidak memiliki akses ke sistem manajemen enterprise yang mahal karena kendala biaya atau kompleksitas.

Masalah umum meliputi:

- **Pelacakan kendaraan yang tidak konsisten** menyebabkan double-booking
- **Pencatatan manual** menyebabkan kehilangan data dan kesalahan
- **Manajemen transaksi yang buruk** mengakibatkan sengketa penagihan
- **Kurangnya pembaruan ketersediaan real-time**
- **Kesulitan dalam mengelola informasi pelanggan dengan aman**

Fokus Solusi:

Proyek ini menyelesaikan masalah-masalah tersebut dengan menyediakan:

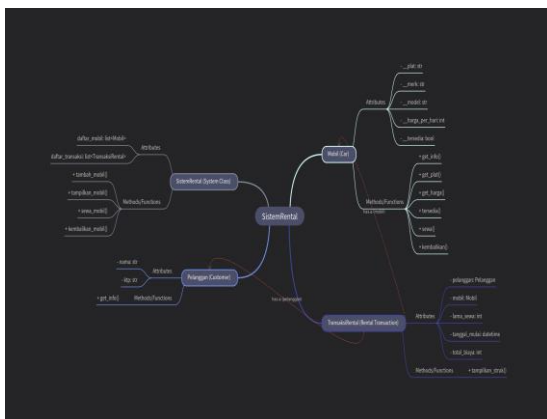
- ✓ **Desain berorientasi objek** mengikuti prinsip SOLID untuk maintainability
- ✓ **Encapsulation** untuk melindungi data sensitif (detail mobil, KTP pelanggan)
- ✓ **Pemrosesan transaksi otomatis** dengan pembuatan struk
- ✓ **Pelacakan ketersediaan real-time** melalui manajemen status
- ✓ **Arsitektur scalable** memungkinkan penambahan fitur dengan mudah
- ✓ **Fondasi untuk peningkatan masa depan** seperti integrasi database dan GUI

Dengan mengimplementasikan solusi ini menggunakan C++ dan prinsip OOP, proyek ini mengatasi kebutuhan bisnis dunia nyata sambil mendemonstrasikan konsep rekayasa perangkat lunak penting dalam desain class, hubungan objek, dan arsitektur sistem.

Desain Sistem

Arsitektur sistem dibangun di sekitar empat class utama yang berinteraksi untuk mengelola alur kerja rental yang lengkap. Desain ini mengikuti praktik terbaik OOP dengan pemisahan concern yang jelas dan hubungan yang terdefinisi dengan baik antar komponen.

Mind Map Sistem Lengkap:



Gambar 1: Arsitektur Sistem Komprehensif menunjukkan semua class, atribut, metode, dan hubungannya

Gambaran Struktur Visual:

Mind map mengilustrasikan struktur OOP yang lengkap:

1. SistemRental (Inti Sistem)

- Mengelola dua komponen utama:
 - **Pelanggan (Customer)** dengan atribut: nama, ktp
 - **TransaksiRental (Rental Transaction)** dengan metode: tampilkan_struk()

2. SistemRental (Class Sistem)

- **Atribut:** daftar_transaksi (list), daftar_mobil (list/table)
- **Metode/Fungsi:**
 - kembalikan_mobil()
 - sewa_mobil()
 - tampilkan_mobil_tersedia()
 - tambah_mobil_baru()

3. Mobil (Car)

- **Metode/Fungsi:**
 - kembalikan()
 - sewa()
 - is_tersedia()
 - get_harga()
- **Atribut:**
 - plat (nomor plat)
 - merk (merek)
 - model
 - harga_per_hari
 - tersedia (status ketersediaan)

Struktur hierarki ini mendemonstrasikan bagaimana semua komponen bekerja bersama untuk membentuk ekosistem manajemen rental yang lengkap.

Struktur Class Inti:

1. Class Mobil

Merepresentasikan kendaraan individual dalam armada rental.

Atribut (Private):

- **string plat** → Nomor plat kendaraan (identifier unik)

- `string merk` → Merek mobil
- `string model` → Model mobil
- `double harga_per_hari` → Harga sewa per hari
- `bool tersedia` → Status ketersediaan

Metode (Public):

- `void get_info()` → Menampilkan informasi mobil
- `string get_plat()` → Mengambil nomor plat
- `double get_harga()` → Mengambil tarif harian
- `void sewa()` → Menandai mobil sebagai disewa
- `void kembalikan()` → Menandai mobil sebagai tersedia
- `bool is_tersedia()` → Mengecek status ketersediaan

2. Class Pelanggan

Merepresentasikan pelanggan yang menyewa kendaraan.

Atribut:

- `string nama` → Nama pelanggan
- `string ktp` → Nomor KTP

Metode:

- `void get_info()` → Menampilkan informasi pelanggan

3. Class TransaksiRental

Merepresentasikan transaksi rental individual yang menghubungkan pelanggan dan mobil.

Atribut:

- `Pelanggan* pelanggan` → Pointer ke objek Pelanggan
- `Mobil* mobil` → Pointer ke objek Mobil
- `int lama_sewa` → Durasi sewa (hari)
- `string tanggal_mulai` → Tanggal mulai
- `double total_biaya` → Total biaya

Metode:

- `void tampilkan_struk()` → Generate dan tampilkan struk

4. Class SistemRental

Controller utama yang mengelola seluruh operasi rental.

Atribut:

- `vector<Mobil*> daftar_mobil` → Vector berisi semua objek Mobil
- `vector<TransaksiRental*> daftar_transaksi` → Vector berisi semua objek TransaksiRental

Metode:

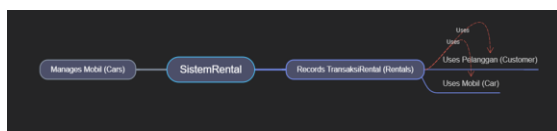
- `void tambah_mobil(Mobil* mobil)` → Menambahkan mobil ke inventaris
- `void tampilkan_mobil()` → Menampilkan mobil yang tersedia

- `void sewa_mobil(string plat, Pelanggan* pelanggan, int lama_sewa)` → Memproses transaksi rental
- `void kembalikan_mobil(string plat)` → Memproses pengembalian mobil

Hubungan Antar Class:

Hubungan	Deskripsi
SistemRental → Mobil	SistemRental <i>memiliki banyak</i> objek Mobil (komposisi)
SistemRental → TransaksiRental	SistemRental <i>mengelola banyak</i> objek TransaksiRental
TransaksiRental → Pelanggan	TransaksiRental <i>melibatkan satu</i> Pelanggan
TransaksiRental → Mobil	TransaksiRental <i>menggunakan satu</i> Mobil untuk rental

Diagram Hubungan Sistem:



Gambar 1: Diagram Hubungan Class menunjukkan interaksi antara SistemRental, TransaksiRental, Pelanggan, dan Mobil

Representasi Visual:

- **SistemRental** (pusat) mengelola Mobil dan mencatat TransaksiRental

- **TransaksiRental** menggunakan Pelanggan (Customer) dan Mobil (Car)
- Alurnya menunjukkan: Manages Mobil (Cars) → SistemRental → Records TransaksiRental (Rentals) → Uses Pelanggan + Uses Mobil

Prinsip Desain Utama:

- **SistemRental** bertindak sebagai controller utama dan entry point
- **TransaksiRental** menghubungkan **Pelanggan** (customer) dan **Mobil** (car)
- **Mobil** dan **Pelanggan** adalah entitas independen yang digunakan oleh **TransaksiRental**
- Encapsulation melindungi data sensitif melalui atribut private

Alur Logika Program

1. Inisialisasi Sistem

- Buat objek **SistemRental** untuk mengelola semua operasi

2. Tambah Mobil ke Inventaris

- Buat beberapa objek **Mobil** dengan detail lengkap
- Tambahkan ke daftar mobil sistem

3. Tampilkan Mobil yang Tersedia

- Sistem menampilkan semua mobil dengan status **tersedia = true**

4. Buat Profil Pelanggan

- Buat objek **Pelanggan** untuk penyewa

5. Proses Rental Mobil

- Panggil `sewa_mobil(plat, pelanggan, lama_sewa)`
- Di dalam metode ini:
 - Sistem mencari mobil berdasarkan nomor plat
 - Cek apakah mobil tersedia
 - Buat objek `TransaksiRental` jika tersedia
 - Ubah status mobil menjadi "tidak tersedia"
 - Generate dan print struk

6. Kembalikan Mobil

- Panggil `kembalikan_mobil(plat)`
- Sistem mencari mobil dan mengubah status menjadi "tersedia"

7. Tampilkan Daftar Terupdate

- Tampilkan semua mobil tersedia dengan status terbaru

Fitur Utama Program

- ✓ **Tambah Mobil ke Inventaris:** Daftarkan kendaraan baru dengan detail lengkap
- ✓ **Lihat Mobil Tersedia:** Tampilkan semua mobil yang tersedia untuk disewa
- ✓ **Registrasi Pelanggan:** Simpan informasi pelanggan dengan aman
- ✓ **Proses Transaksi Rental:** Buat catatan rental dengan kalkulasi biaya otomatis
- ✓ **Generate Struk:** Print detail transaksi untuk catatan pelanggan

✓ **Kembalikan Mobil:** Update ketersediaan mobil setelah pengembalian

✓ **Encapsulation:** Lindungi data sensitif menggunakan atribut private

✓ **Hubungan Objek:** Demonstrasikan pola composition dan association

✓ **Dynamic Memory:** Gunakan pointer dan dynamic allocation untuk fleksibilitas

Contoh Implementasi

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

// Class Mobil
class Mobil {
private:
    string plat;
    string merk;
    string model;
    double harga_per_hari;
    bool tersedia;

public:
    // Constructor
    Mobil(string p, string m, string mdl,
           double harga)
        : plat(p), merk(m), model(mdl),
          harga_per_hari(harga), tersedia(true) {}

    // Getter methods
    string get_plat() { return plat; }
    double get_harga() { return
harga_per_hari; }
    bool is_tersedia() { return tersedia; }

    // Display info
    void get_info() {
        cout << "Plat: " << plat << " | Merk:
" << merk
              << " | Model: " << model << " |
Harga: Rp" << harga_per_hari
```

```

        << "/hari | Status: " << (tersedia
? "Tersedia" : "Disewa") << endl;
    }

    // Rental operations
    void sewa() { tersedia = false; }
    void kembalikan() { tersedia = true; }
};

// Class Pelanggan
class Pelanggan {
public:
    string nama;
    string ktp;

    Pelanggan(string n, string k) :
nama(n), ktp(k) {}

    void get_info() {
        cout << "Nama: " << nama << " |
KTP: " << ktp << endl;
    }
};

// Class TransaksiRental
class TransaksiRental {
private:
    Pelanggan* pelanggan;
    Mobil* mobil;
    int lama_sewa;
    string tanggal_mulai;
    double total_biaya;

public:
    TransaksiRental(Pelanggan* p,
Mobil* m, int lama, string tanggal)
        : pelanggan(p), mobil(m),
lama_sewa(lama),
tanggal_mulai(tanggal) {
        total_biaya = lama_sewa * mobil-
>get_harga();
    }

    void tampilkan_struk() {
        cout << "\n===== STRUK
RENTAL MOBIL =====" << endl;
        cout << "Pelanggan: " <<
pelanggan->nama << endl;
        cout << "Mobil: " << mobil-
>get_plat() << endl;
        cout << "Tanggal Mulai: " <<
tanggal_mulai << endl;
        cout << "Lama Sewa: " <<
lama_sewa << " hari" << endl;
    }
};

```

```

        cout << "Total Biaya: Rp" <<
total_biaya << endl;
        cout <<
"=====
=====\\n" << endl;
    }
};

// Class SistemRental
class SistemRental {
private:
    vector<Mobil*> daftar_mobil;
    vector<TransaksiRental*>
daftar_transaksi;

public:
    void tambah_mobil(Mobil* mobil) {
        daftar_mobil.push_back(mobil);
        cout << "Mobil berhasil
ditambahkan!" << endl;
    }

    void tampilkan_mobil() {
        cout << "\\n=== DAFTAR MOBIL
TERSEDIA ===" << endl;
        for (auto mobil : daftar_mobil) {
            if (mobil->is_tersedia()) {
                mobil->get_info();
            }
        }
        cout << endl;
    }

    void sewa_mobil(string plat,
Pelanggan* pelanggan, int lama_sewa)
    {
        for (auto mobil : daftar_mobil) {
            if (mobil->get_plat() == plat &&
mobil->is_tersedia()) {
                mobil->sewa();
                TransaksiRental* transaksi =
new TransaksiRental(
                    pelanggan, mobil,
                    lama_sewa, "30-12-2024"
                );

                daftar_transaksi.push_back(transaksi);
                transaksi->tampilkan_struk();
                return;
            }
        }
        cout << "Mobil tidak tersedia atau
tidak ditemukan!" << endl;
    }
};

```

```

void kembalikan_mobil(string plat) {
    for (auto mobil : daftar_mobil) {
        if (mobil->get_plat() == plat) {
            mobil->kembalikan();
            cout << "Mobil " << plat << "
berhasil dikembalikan!" << endl;
            return;
        }
    }
    cout << "Mobil tidak ditemukan!" <<
endl;
}
};

// Main Program
int main() {
    // 1. Inisialisasi sistem
    SistemRental sistem;

    // 2. Tambah mobil ke inventaris
    Mobil* mobil1 = new
Mobil("B1234XYZ", "Toyota", "Avanza",
350000);
    Mobil* mobil2 = new
Mobil("B5678ABC", "Honda", "Jazz",
400000);
    Mobil* mobil3 = new
Mobil("B9012DEF", "Suzuki", "Ertiga",
380000);

    sistem.tambah_mobil(mobil1);
    sistem.tambah_mobil(mobil2);
    sistem.tambah_mobil(mobil3);

    // 3. Tampilkan mobil tersedia
    sistem.tampilkan_mobil();

    // 4. Buat pelanggan
    Pelanggan* pelanggan1 = new
Pelanggan("Budi Santoso",
"3201234567890123");

    // 5. Sewa mobil
    sistem.sewa_mobil("B1234XYZ",
pelanggan1, 3);

    // 6. Tampilkan mobil tersedia setelah
rental
    sistem.tampilkan_mobil();

    // 7. Kembalikan mobil
    sistem.kembalikan_mobil("B1234XYZ");

```

```

// 8. Tampilkan mobil tersedia setelah
pengembalian
sistem.tampilkan_mobil();

return 0;
}

```

Struktur Data yang Digunakan

Vector (STL Container):

- `vector<Mobil*>` untuk menyimpan daftar mobil
- `vector<TransaksiRental*>` untuk menyimpan daftar transaksi
- Dynamic array yang bisa berkembang sesuai kebutuhan
- Akses cepat dengan index

Pointer:

- Digunakan untuk hubungan antar objek
- Memory management dengan `new` dan `delete`
- Memungkinkan polymorphism dan dynamic allocation

String (STL):

- Untuk menyimpan data teks (nama, plat, model)
- Built-in functions untuk manipulasi string

Contoh Output Program

Menambahkan Mobil:

Output

```
Mobil berhasil ditambahkan!  
Mobil berhasil ditambahkan!  
Mobil berhasil ditambahkan!
```

Menampilkan Daftar Mobil:

```
Output Clear  
==== DAFTAR MOBIL TERSEDIA ====  
Plat: B1234XYZ | Merk: Toyota | Model: Avanza | Harga: Rp350000  
/hari | Status: Tersedia  
Plat: B5678ABC | Merk: Honda | Model: Jazz | Harga: Rp400000/hari  
| Status: Tersedia  
Plat: B9012DEF | Merk: Suzuki | Model: Ertiga | Harga: Rp380000  
/hari | Status: Tersedia
```

Menyewa Mobil:

```
Output  
===== STRUK RENTAL MOBIL =====  
Pelanggan: Budi Santoso  
Mobil: B1234XYZ  
Tanggal Mulai: 30-12-2024  
Lama Sewa: 3 hari  
Total Biaya: Rp1.05e+06  
=====
```

Mengembalikan Mobil:

```
Output  
Mobil B1234XYZ berhasil dikembalikan!
```

Tantangan dan Pembelajaran

Tantangan Utama:

1. Memory Management:

- Alokasi dan dealokasi memory dengan benar sangat penting
- Penggunaan `new` dan `delete` untuk objek dinamis

- Mencegah memory leak dan dangling pointer
- Memahami kapan menggunakan pointer vs reference

2. Manajemen Hubungan Objek:

- Memastikan komposisi yang tepat antara SistemRental dan Mobil
- Mengelola referensi objek dalam TransaksiRental
- Menghindari circular dependency
- Pointer vs object ownership

3. Manajemen State:

- Menjaga konsistensi status ketersediaan mobil
- Mencegah skenario double-booking
- Menangani edge cases (menyewa mobil tidak tersedia, mengembalikan mobil tidak ada)
- Sinkronisasi data antar class

4. Integritas Data:

- Validasi keunikan nomor plat
- Memastikan kelengkapan data pelanggan
- Kalkulasi biaya yang akurat
- Input validation dan error handling

5. String Handling dalam C++:

- Menggunakan `std::string` dengan benar
- Handling input dengan `getline()` untuk nama dengan spasi
- Membersihkan input buffer
- String comparison dan manipulation

Pembelajaran yang Didapat:

✓ Pentingnya prinsip OOP dalam membuat kode yang maintainable dan scalable

✓ **Manfaat encapsulation** untuk proteksi data dan controlled access

✓ **Pattern desain class** untuk merepresentasikan entitas bisnis dunia nyata

✓ **Object composition** untuk membangun sistem kompleks dari komponen sederhana

✓ **Defensive programming** untuk menangani input tidak valid dan edge cases

✓ **Memory management di C++** - pentingnya manual allocation/deallocation

✓ **STL containers** (vector) untuk struktur data dinamis

✓ **Pointer manipulation** dan pentingnya memahami memory address

Pengembangan Masa Depan

Prototype ini dapat dikembangkan menjadi sistem yang lebih advanced dengan fungsionalitas tambahan:

1. Integrasi Database

- Gunakan SQLite atau MySQL untuk penyimpanan data persisten
- Memungkinkan pengambilan data antar sesi
- Support query dan reporting advanced
- Backup dan recovery data

2. Autentikasi User

- Tambah sistem login admin dan pelanggan
- Implementasi role-based access control
- Secure operasi sensitif dengan password
- Session management

3. Pemrosesan Pembayaran

- Integrasi payment gateway API
- Support berbagai metode pembayaran
- Generate invoice digital
- History pembayaran

4. Booking Advance

- Izinkan reservasi untuk tanggal masa depan
- Implementasi sistem kalender
- Kirim konfirmasi booking via email/SMS
- Reminder otomatis

5. Graphical User Interface (GUI)

- Build desktop GUI menggunakan Qt atau wxWidgets
- Buat interface web menggunakan web framework
- Develop mobile app dengan cross-platform tools
- User-friendly dashboard

6. Reporting dan Analytics

- Generate laporan revenue
- Track model mobil populer
- Analisis pola dan trend rental
- Visualisasi data dengan charts
- Export ke PDF/Excel

7. Denda Keterlambatan

- Kalkulasi otomatis denda terlambat
- Implementasi grace period
- Notifikasi reminder

- History keterlambatan pelanggan

8. Tracking Maintenance Mobil

- Jadwal maintenance berdasarkan mileage/waktu
- Tandai mobil sebagai "under maintenance"
- Track service history
- Notifikasi service due
- Cost tracking untuk maintenance

9. Support Multi-Branch

- Kelola multiple lokasi rental
- Enable transfer mobil antar cabang
- Centralized inventory management
- Branch-wise reporting

10. Program Loyalitas Pelanggan

- Implementasi sistem poin reward
- Tawarkan diskon untuk penyewa frequent
- Track rental history
- Member tiers (Silver, Gold, Platinum)
- Special promotions

11. File I/O untuk Persistensi

- Simpan dan load data dari file
- Export/import data dalam format CSV
- Configuration files
- Backup system

12. Error Handling yang Robust

- Try-catch blocks untuk exception handling
- Custom exception classes
- Logging system untuk debugging
- User-friendly error messages

Pengembangan ini akan secara signifikan meningkatkan fungsionalitas, user

experience, scalability, dan kesiapan untuk deployment komersial.

Kesimpulan dan Refleksi

Proyek ini berhasil mendemonstrasikan bagaimana prinsip Object-Oriented Programming dapat diterapkan untuk menyelesaikan masalah bisnis dunia nyata di industri rental mobil. Sistem ini secara efektif mengelola operasi inti sambil mempertahankan kode yang clean, modular, dan maintainable.

Pencapaian Utama:

✓ **Implementasi OOP fungsional** dengan desain class yang tepat

✓ **Pemisahan concern yang jelas** melalui class yang well-defined

✓ **Encapsulation** melindungi data bisnis sensitif

✓ **Arsitektur scalable** siap untuk pengembangan masa depan

✓ **Real-world problem solving** mengatasi kebutuhan bisnis aktual

✓ **Memory management** yang proper dengan dynamic allocation

✓ **STL usage** untuk struktur data yang efisien

Nilai Edukatif:

Proyek ini memperkuat konsep fundamental dalam:

- Desain dan instantiasi class

- Hubungan objek (composition, association)
- Encapsulation dan data hiding
- Desain dan implementasi method
- Arsitektur sistem dan desain workflow
- Memory management di C++
- Pointer dan reference
- STL containers dan algorithms

Meskipun implementasi saat ini terlihat basic, ini menyediakan fondasi solid untuk sistem manajemen rental komersial. Desain modular memastikan bahwa fitur baru dapat ditambahkan tanpa refactoring major, mendemonstrasikan kekuatan dan fleksibilitas desain berorientasi objek.

Catatan Akhir:

Sistem yang dihasilkan adalah aplikasi fungsional yang memenuhi tujuan desainnya dan mendemonstrasikan best practices dalam programming C++. Ini berfungsi sebagai contoh excellent bagaimana konsep programming fundamental dapat menciptakan solusi yang bermakna dan praktis untuk operasi bisnis.

Proyek ini telah memberikan pengalaman hands-on yang berharga dalam menerapkan prinsip OOP ke skenario dunia nyata, menekankan pentingnya planning, desain modular, dan berpikir tentang scalability masa depan sejak awal.

Objects	<pre> mobil1 = Mobil(...), pelanggan1 = Pelanggan(...), sistem = SistemRental() </pre>
Alur Logika	Tambah mobil → Tampilkan daftar → Buat pelanggan → Sewa mobil → Generate struk → Kembalikan mobil
Prinsip OOP	Encapsulation, Composition, Association, Abstraction
Bahasa	C++ dengan STL
Struktur Data	Vector, Pointer, String

Tabel Ringkasan

Konsep	Contoh
Classes	Mobil, Pelanggan, TransaksiRental, SistemRental